

LAPORAN PRAKTIKUM INDIVIDU
PERTEMUAN 5
TEKNOLOGI IOT II

Disusun dalam rangka memenuhi salah satu syarat kelengkapan nilai Praktikum
Mata Kuliah Teknologi IoT II

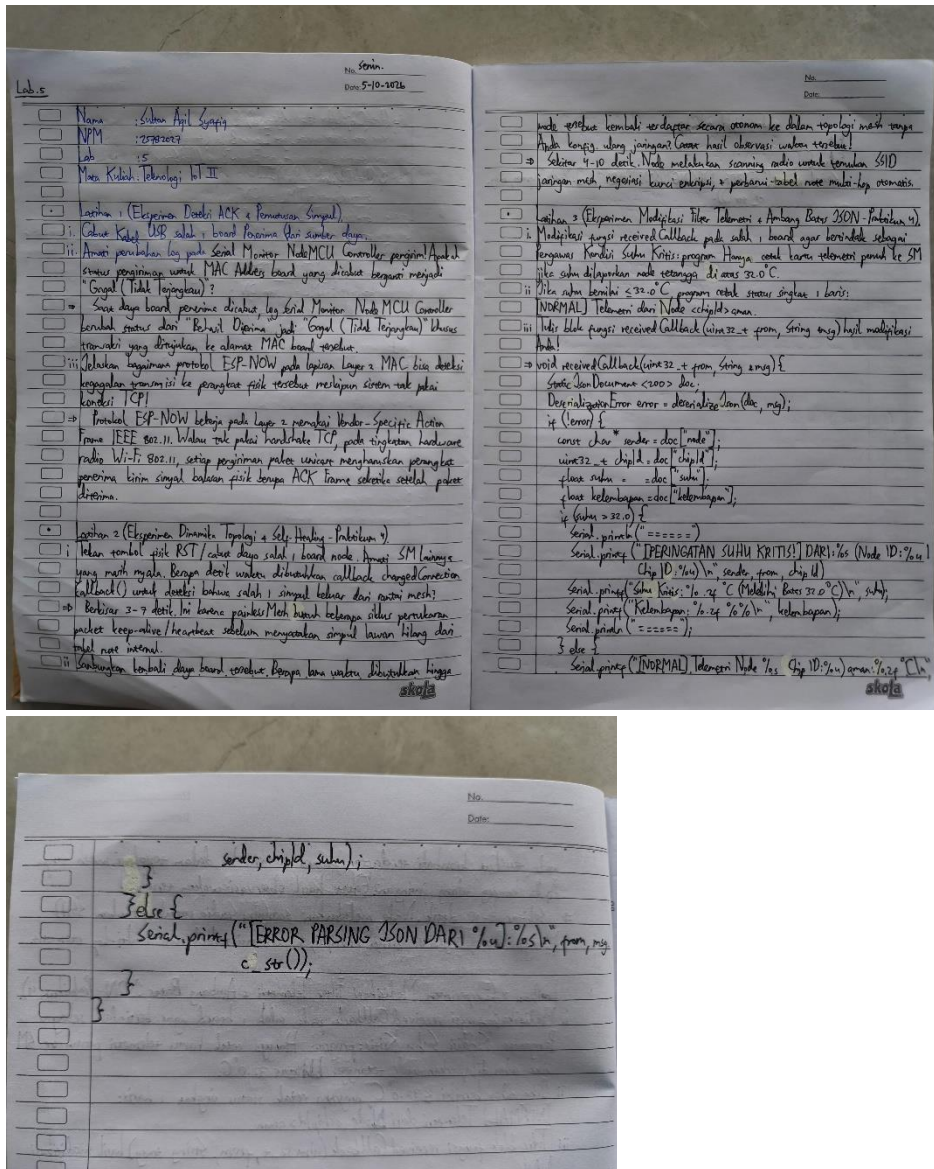


DISUSUN OLEH :

NAMA : SULTAN AQIL SYAFIQ.
NPM : 25782027
PRODI : TEKNOLOGI REKAYASA INTERNET
KELAS : 3A
DOSEN : Dr.Ir. Septafiansyah Dwi Putra, S.T., M.T.
Agus Ambarwari, S.Pd., M.Kom.

KEMENTERIAN PENDIDIKAN TINGGI, SAINS, DAN
TEKNOLOGI
POLITEKNIK NEGERI LAMPUNG
TAHUN AJARAN 2026 / 2027

❖ **LOGBOOK:**



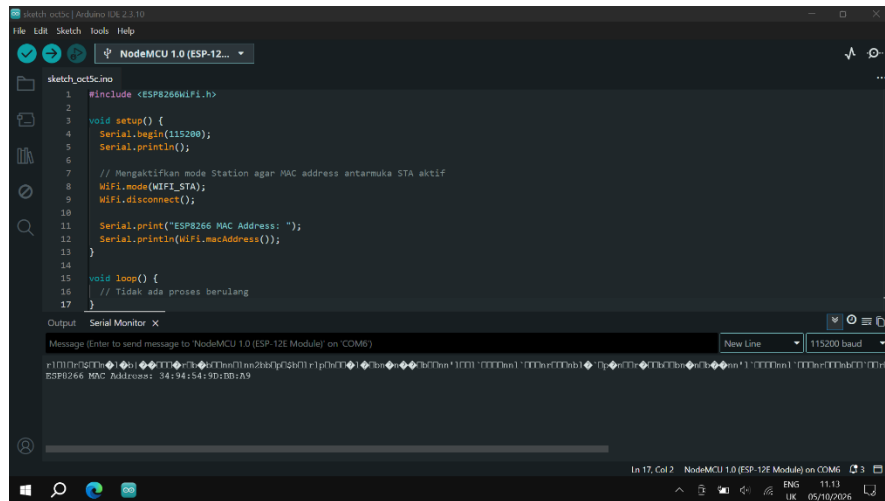
❖ **Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU.**

Langkah Kerja Rangkaian:

1. Buat berkas baru (*New Sketch*) pada Arduino IDE.
2. Tuliskan kode program pembaca MAC Address.
3. **Upload** program ke masing-masing *board* NodeMCU dalam kelompok Anda secara bergantian.
4. Buka **Serial Monitor** pada kecepatan **115200 baud**, lalu tekan tombol fisik **RST** pada *board*.
5. Catat 6 pasang digit heksadesimal yang muncul di layar (contoh: BC:FF:4D:23:45:90), dan beri label tiap mac address masing – masing NodeMCU.

- SCREENSHOT PRAKTIKUM 1:

- *Serial Monitor:*



```

1 #include <ESP8266WiFi.h>
2
3 void setup() {
4   Serial.begin(115200);
5   Serial.println();
6
7   // Mengaktifkan mode Station agar MAC address antarmuka STA aktif
8   WiFi.mode(WIFI_STA);
9   WiFi.disconnect();
10
11   Serial.print("ESP8266 MAC Address: ");
12   Serial.println(WiFi.macAddress());
13 }
14
15 void loop() {
16   // Tidak ada proses berulang
17 }
  
```

Output: Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM6')

ESP8266 MAC Address: 34:94:54:9D:BB:A9

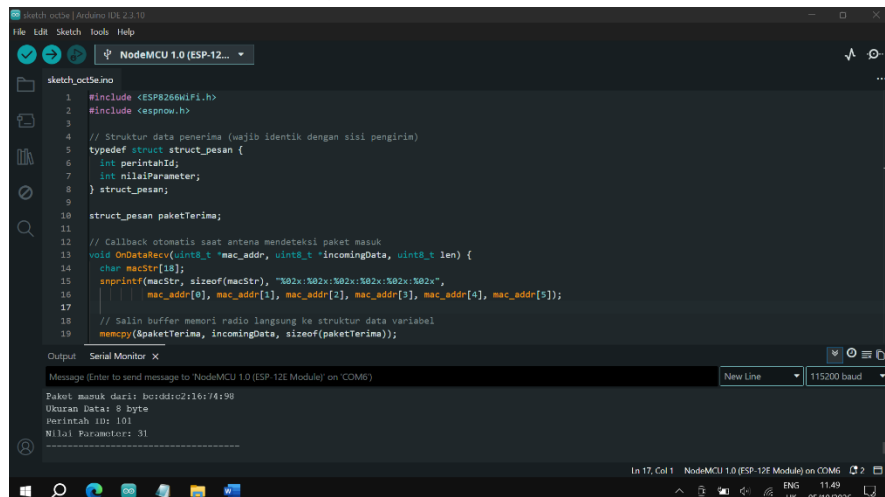
❖ Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW).

Langkah Kerja Rangkaian:

1. Unggah kode sketsa berikut ke NodeMCU 2 dan NodeMCU 3.
2. Board penerima tidak perlu mengetahui alamat MAC milik Controller.

- SCREENSHOT PRAKTIKUM 2:

- *Serial Monitor (Klien):*



```

1 #include <ESP8266WiFi.h>
2 #include <ESPNow.h>
3
4 // Struktur data penerima (wajib identik dengan sisi pengirim)
5 typedef struct struct_pesanan {
6   int perintahId;
7   int nilaiParameter;
8 } struct_pesanan;
9
10 struct_pesanan paketTerima;
11
12 // Callback otomatis saat antena mendeteksi paket masuk
13 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
14   char macStr[18];
15   snprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x",
16            mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
17
18   // Salin buffer memori radio langsung ke struktur data variabel
19   memcpy(&paketTerima, incomingData, sizeof(paketTerima));
  
```

Output: Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM6')

Paket masuk dari: bc:dd:c2:16:74:98
 Ukuran data: 8 byte
 Perintah ID: 101
 Nilai Parameter: 31

- *Serial Monitor (Server):*

```

1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // Ganti dengan MAC Address Board Penerima 1 dan board Penerima 2 Anda!
5 uint8_t receiver1[] = {0x14, 0x04, 0x04, 0x00, 0x00, 0x00};
6 uint8_t receiver2[] = {0x02, 0x00, 0x00, 0x02, 0x03, 0x00};
7 uint8_t receiver3[] = {0x08, 0x08, 0x04, 0x09, 0x0B, 0x00};
8 uint8_t receiver4[] = {0x09, 0x00, 0x03, 0x11, 0x24, 0x00};
9
10 // Definisi struktur data paket instruksi
11 typedef struct struct_pesan {
12   int perintahId;
13   int nilaiParameter;
14 } struct_pesan;
15
16 struct_pesan paketKirim;
17
18 unsigned long previousMillis = 0;
19 const long interval = 2000; // Transmisi berkala setiap 2 detik
20
21 // callback otomatis saat paket selesai dipancarkan radio

```

Serial Monitor X

```

11:29:15.619 -> Kirim paket ke: 34:94:54:9d1bb4a9 | Status: Berhasil Diterima
11:29:15.619 -> Kirim paket ke: 8c5a1b5f62c31ac | Status: Berhasil Diterima
11:29:17.404 -> Kirim paket ke: 34:94:54:9d1bb4a9 | Status: Berhasil Diterima
11:29:17.404 -> Kirim paket ke: 8c5a1b5f62c31ac | Status: Berhasil Diterima
11:29:19.607 -> Kirim paket ke: 34:94:54:9d1bb4a9 | Status: Berhasil Diterima
11:29:19.607 -> Kirim paket ke: 8c5a1b5f62c31ac | Status: Berhasil Diterima
11:29:21.428 -> Kirim paket ke: 34:94:54:9d1bb4a9 | Status: Berhasil Diterima
11:29:21.428 -> Kirim paket ke: 8c5a1b5f62c31ac | Status: Berhasil Diterima
11:29:23.602 -> Kirim paket ke: 34:94:54:9d1bb4a9 | Status: Berhasil Diterima
11:29:23.602 -> Kirim paket ke: 8c5a1b5f62c31ac | Status: Berhasil Diterima

```

❖ Praktikum 3: Pengumpulan Data Telemetry Many-to-One (ESP-NOW).

Langkah Kerja Rangkaian:

1. Unggah kode sketsa berikut ke NodeMCU 2 dan NodeMCU 3.
2. Board penerima tidak perlu mengetahui alamat MAC milik Controller.

• SCREENSHOT PRAKTIKUM 3:

- *Serial Monitor (Klien):*

```

1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 typedef struct struct_sensor {
5   int id;
6   float pembacaan1;
7   float pembacaan2;
8 } struct_sensor;
9
10 struct_sensor dataTerima;
11
12 // Array penampung data terakhir: Index 0 untuk Node 1, Index 1 untuk Node 2
13 struct_sensor records[2];
14
15 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
16   memcpy(&dataTerima, incomingData, sizeof(dataTerima));
17
18   // Validasi ID agar tidak terjadi error index array out-of-bounds
19   if (dataTerima.id >= 1 && dataTerima.id <= 2) {
20     int index = dataTerima.id - 1;

```

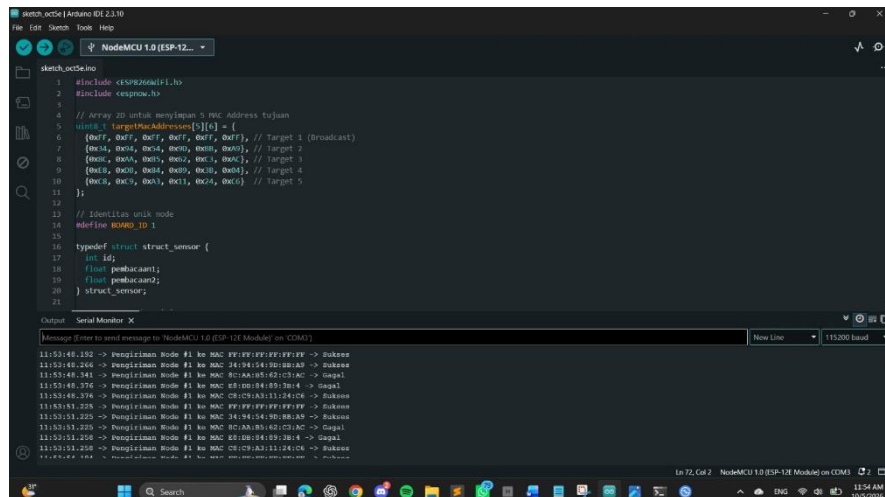
Serial Monitor X

```

DATA TERIMA DARI NODE #1
Parameter 1 : 35.90
Parameter 2 : 40.80

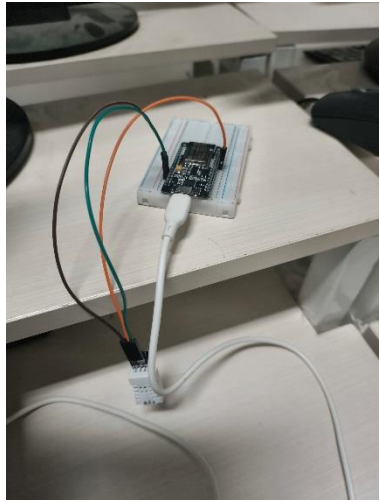
```

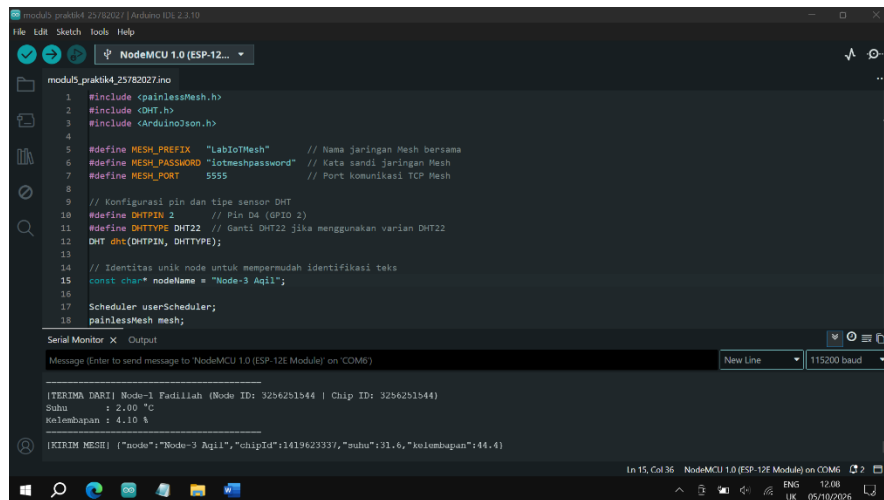
- *Serial Monitor (Server):*



Langkah Kerja Rangkaian:

- SCREENSHOT PRAKTIKUM 4:





```
modul5_praktik4_25782027.ino
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "LabIoTMesh" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "iotmeshpassword" // Kata sandi jaringan Mesh
7 #define MESH_PORT 5555 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-3 Aqil";
16
17 Scheduler userScheduler;
18 painlessMesh mesh;
```

Serial Monitor X Output

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM8')

115200 baud

[TERIMA DARI] Node-1 Fadillah (Node ID: 3256251544 | Chip ID: 3256251544)

Suhu : 2.00 °C

Kelembapan : 4.10 %

[KIRIM MESH] ("node":"Node-3 Aqil","chipId":1419623337,"suhu":31.6,"kelembapan":44.4)

❖ TUGAS MANDIRI:

- Kembangkan jaringan mesh pada Praktikum 4 menjadi sistem *monitoring* lingkungan dan kendali otomasi terdistribusi tanpa perantara router.
- **Skenario Proyek (*Smart Lighting*):** Anda akan mengeksplorasi penggunaan modulasi sinyal listrik analogWrite() (atau dikenal dengan fitur PWM) dan memadukannya dengan teknologi masukan elemen *Slider / Range* dari HTML5.
 - **Tujuan:** Menguasai transmisi paket JSON terdistribusi melintasi jaringan *Ad-Hoc Mesh Multi-Hop* dan mengevaluasi *rule engine* otomasi aktuator LED di node konsentrator.
 - **Tingkat Kesulitan:** Lanjut (Kolaboratif Kelompok)
 - **Instruksi Perangkaian:**
 - i). Node 1 (Stasiun Lingkungan): Hubungkan pin DATA sensor DHT11/DHT22 ke pin D4 (GPIO 2) NodeMCU 1 (VCC ke 3V3, GND ke GND).
 - ii). Node 2 (Stasiun Cahaya): Rangkai sensor LDR dan Resistor 10k Ohm sebagai pembagi tegangan ke pin A0 NodeMCU 2.
 - iii). Node 3 (Hub Kontrol Aktuator):
 - Hubungkan kaki Anoda (kaki panjang) LED ke pin D6 (GPIO 12) NodeMCU 3.
 - Hubungkan kaki Katoda (kaki pendek) LED ke salah satu kaki Resistor 220 Ohm, lalu hubungkan kaki resistor lainnya ke jalur GND (rangkaian LED aktif-HIGH seperti pada Modul 3 dan Modul 4).

- **Instruksi Pemrograman (Front-End & Back-End Mesh):**
 - i). **Node 1 (Sensor DHT):**
 - Modifikasi fungsi `sendMessage()` agar membaca suhu dan kelembapan riil menggunakan pustaka `DHT.h` setiap 3 detik.
 - Kemas data ke dalam format string JSON ringkas dan siarkan via mesh:
`{"tipe":"suhu_node","suhu":29.4,"kelembapan":65.0}`
 - ii). **Node 2 (Sensor LDR):**
 - Modifikasi fungsi `sendMessage()` agar membaca nilai ADC analog LDR setiap 3 detik.
 - Kemas data ke dalam format string JSON ringkas dan siarkan via mesh:
`{"tipe":"cahaya_node","adc":420}`
 - iii). **Node 3 (Hub Aktuator Cerdas):**
 - Nonaktifkan `taskSendMessage` pada Node 3 (node ini bertindak pasif sebagai penerima dan pengambil keputusan).
 - Pada fungsi `receivedCallback(uint32_t from, String &msg)`:
- Gunakan pustaka `ArduinoJson` (`JsonDocument` / `StaticJsonDocument<200>`) untuk membedah string JSON masuk.
- Evaluasi kondisi Rule Engine:
 - JIKA data suhu dari Node 1 bernilai di atas 31.0 Celcius ATAU nilai ADC cahaya dari Node 2 bernilai di bawah 300 (kondisi gelap), nyalakan LED (`digitalWrite(ledPin, HIGH)`).
 - Jika kedua parameter kembali ke kondisi normal, matikan LED (`digitalWrite(ledPin, LOW)`).
- iv). **Eksperimen Ketahanan Jaringan (Multi-Hop dan Self-Healing):**
 - Jauhkan Node 1 dan Node 3 hingga berada di luar jangkauan radio langsung masing-masing.
 - Tempatkan Node 2 di posisi tengah antara Node 1 dan Node 3.

- Amati pada Serial Monitor apakah paket telemetry dari Node 1 tetap dapat diterima oleh Node 3 melalui relai perantara (multi-hop relay) Node 2.
- Matikan daya Node 2 (simulasi node putus/gagal), lalu amati perubahan status pada Serial Monitor Node 3 melalui callback `changedConnectionCallback()`.
- **Kriteria Keberhasilan:**
 - Jauhkan Node 1 dan Node 3 hingga berada di luar jangkauan radio langsung masing-masing.
 - Tempatkan Node 2 di posisi tengah antara Node 1 dan Node 3.
 - Amati pada Serial Monitor apakah paket telemetry dari Node 1 tetap dapat diterima oleh Node 3 melalui relai perantara (multi-hop relay) Node 2.
 - Matikan daya Node 2 (simulasi node putus/gagal), lalu amati perubahan status pada Serial Monitor Node 3 melalui callback `changedConnectionCallback()`.

❖ KODE TUGAS MANDIRI:

```
#include <painlessMesh.h>
#include <ArduinoJson.h>

#define MESH_PREFIX    "LabIoTMesh"
#define MESH_PASSWORD  "iotmeshpassword"
#define MESH_PORT      5555

#define LDRPIN A0      // Pin Analog

Scheduler userScheduler;
painlessMesh mesh;

void sendMessage();
Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);

void sendMessage() {
    int nilaiADC = analogRead(LDRPIN);

    // Format JSON sesuai permintaan tugas
    StaticJsonDocument<200> doc;
    doc["tipe"] = "cahaya_node";
    doc["adc"] = nilaiADC;
```

```

String msg;
serializeJson(doc, msg);
mesh.sendBroadcast(msg);
Serial.print("[Node 2 KIRIM] ");
Serial.println(msg);
}

void setup() {
  Serial.begin(115200);
  mesh.setDebugMsgTypes(ERROR | STARTUP);
  mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);
  userScheduler.addTask(taskSendMessage);
  taskSendMessage.enable();
}

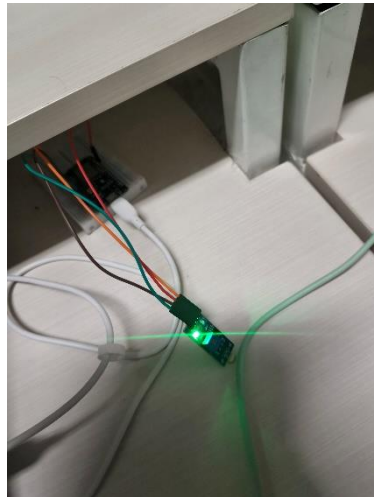
void loop() {
  mesh.update();
}

```

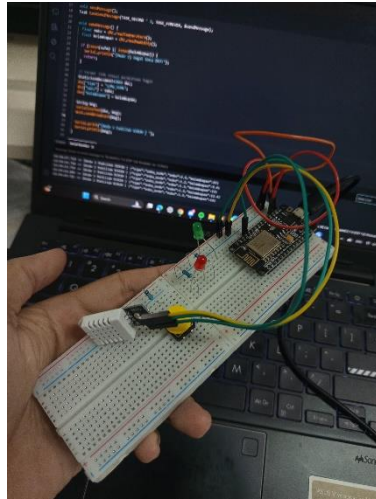
❖ **SCREENSHOT TUGAS MANDIRI:**

- *Foto rangkaian:*

➤ LDR:



➤ DHT22:



➤ LED:



- *Serial Monitor:*

➤ LDR:

```

1 #include <painlessMesh.h>
2 #include <ArduinoJson.h>
3
4 #define MESH_PREFIX "LabIoTMesh"
5 #define MESH_PASSWORD "Iotmeshpassword"
6 #define MESH_PORT 5555
7
8 #define LDRPIN A0 // Pin Analog
9
10 Scheduler userScheduler;
11 painlessMesh mesh;
12
13 void sendMessage();
14 Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
15
16 void sendMessage() {
17   int nilaiADC = analogRead(LDRPIN);
18   Serial Monitor X
19 }

```

Output Serial Monitor X

```

[Node 2 KIRIM] ("type":"cahaya_node","adc":205)
[Node 2 KIRIM] ("type":"cahaya_node","adc":160)
[Node 2 KIRIM] ("type":"cahaya_node","adc":142)
[Node 2 KIRIM] ("type":"cahaya_node","adc":286)
[Node 2 KIRIM] ("type":"cahaya_node","adc":669)
[Node 2 KIRIM] ("type":"cahaya_node","adc":446)
[Node 2 KIRIM] ("type":"cahaya_node","adc":838)

```

➤ DHT22:

```

17 void sendMessage();
18 Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
19
20 void sendMessage() {
21   float suhu = dht.readTemperature();
22   float kelembapan = dht.readHumidity();
23
24   if (isnan(suhu) || isnan(kelembapan)) {
25     Serial.println("[Node 1] gagal baca DHT!");
26     return;
27   }
28
29   // Format JSON sesuai permintaan tugas
30   StaticJsonDocument<200> doc;
31   doc["type"] = "suhu_node";
32   doc["suhu"] = suhu;
33   doc["kelembapan"] = kelembapan;
34
35   String msg;
36   serializeJson(doc, msg);
37   mesh.sendMessage(msg);
38
39   Serial.print("[Node 1 Fadillah KIRIM ] ");
40   Serial.println(msg);
41 }
42

```

Output Serial Monitor X

```

Message (Enter to send message to NodeMCU 1.0 (ESP-12E Module) on COM1)
12:34:07.719 -> [Node 1 Fadillah KIRIM ] {"type":"suhu_node","suhu":2.5,"kelembapan":20}
12:34:10.736 -> [Node 1 Fadillah KIRIM ] {"type":"suhu_node","suhu":1.2,"kelembapan":9.4}
12:34:13.704 -> [Node 1 Fadillah KIRIM ] {"type":"suhu_node","suhu":1.3,"kelembapan":13}
12:34:16.699 -> [Node 1 Fadillah KIRIM ] {"type":"suhu_node","suhu":1.3,"kelembapan":12.6}
12:34:19.721 -> [Node 1 Fadillah KIRIM ] {"type":"suhu_node","suhu":1.3,"kelembapan":16}

```

➤ LED:

```

47 Serial.print("[KIRIM RESP] ");
48 Serial.println(msg);
49
50 }
51
52 // callback otomatis saat menerima pesan dari node maupun di jaringan mesh
53 void receivedCallback(uint32_t from, String msg) {
54   // parsing pesan teks json yang diterima
55   StaticJsonDocument<200> doc;
56   DeserializationError error = deserializeJson(doc, msg);
57
58   if (error) {
59     const char* sender = doc["node"];
60     uint32_t chipid = doc["chipid"];
61     float suhu = doc["suhu"];
62     float kelembapan = doc["kelembapan"];
63
64     Serial.println("=====");
65     Serial.print("[TERIMA DATA] <[Node ID: <u> <u> | Chip ID: <u>]\n", sender, from, chipid);
66     Serial.print("Suhu : <f> %\n", suhu);
67     Serial.print("Kelembapan : <f> %\n", kelembapan);
68     Serial.println("=====");
69   } else {
70     Serial.print("[TERIMA DATA MENJADI <u>]: <f>\n", from, msg.c_str());
71   }
72 }
73
74 // Callback otomatis saat ada perangkat baru bergabung ke mesh
75

```

Output Serial Monitor X

```

Message (Enter to send message to NodeMCU 1.0 (ESP-12E Module) on COM1)
12:35:26.554 -> [TERIMA DATA] Suhu: 1.00 °C | STATUS: LED MATI (Normal)
12:35:29.174 -> [TERIMA DATA] Cahaya (ADC): 859 | STATUS: LED MATI (Normal)
12:35:29.578 -> [TERIMA DATA] Suhu: 1.00 °C | STATUS: LED MATI (Normal)
12:35:32.260 -> [TERIMA DATA] Cahaya (ADC): 887 | STATUS: LED MATI (Normal)
12:35:32.673 -> [TERIMA DATA] Suhu: 2.50 °C | STATUS: LED MATI (Normal)

```